

[DHS/2025/Q3]

3 Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

A container yard is the main area in a shipping port where shipping containers are stored. This is a large, organised space where containers are stacked and temporarily stored.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 3.1  
        Program code
```

Output:

Task 3.1

A shipping port is planning to create a program to manage `ContainerNode` via `LinkedList` Data Structure. Each `ContainerNode` represents a shipping container.

The class `ContainerNode` contains four **private** attributes:

- `container_number` is the ID of the shipping container, for example, MSKU1234567
- `category_of_goods` is the type of goods stored, for example, Electronics
- `cargo_weight` is the weight of the goods stored, for example, 15000 kg
- `next` is the pointer that points to the next `ContainerNode`, initialised to `None`

The class `LinkedList` contains one **private** attribute:

- `head` is the pointer that points to the first `ContainerNode`, initialised to `None`

The class `LinkedList` has the following methods:

- `insert_to_end()` takes the `container_number`, `category_of_goods` and `cargo_weight` as parameters, creates a new `ContainerNode` instance and inserts it at the end of the `LinkedList`.
- `insert_to_front()` takes the `container_number`, `category_of_goods` and `cargo_weight` as parameters, creates a new `ContainerNode` instance and inserts it at the front of the `LinkedList`.
- `delete_from_end()` removes the last `ContainerNode` instance from the `LinkedList` and returns the deleted `ContainerNode` instance (to return string "Cannot delete - Empty" if `LinkedList` is empty)
- `delete_from_front()` removes the first `ContainerNode` instance in the `LinkedList` and returns the deleted `ContainerNode` instance (to return string "Cannot delete - Empty" if `LinkedList` is empty)

Write program code to declare the class `ContainerNode` and `LinkedList`. Ensure that each class contains a complete set of methods, including all the accessors and mutators. [13]

[DHS/2025/Q3 Cont ']

Task 3.2

The shipping port decided to manage the `ContainerNode` via `Stack` Data Structure instead.

Write a Python subclass `Stack` using `LinkedList` as its superclass. The subclass `Stack` has the following additional methods:

- `is_empty()` returns `True` if the `Stack` is empty, otherwise `False`
- `peek()` returns the `container_number` of the topmost `ContainerNode` instance. Return "No containers" if `Stack` is empty
- `display()` shows the `container_number` of all `ContainerNode` instances in the `Stack` using the below format (Note: the most recently added `ContainerNode` appears at the top). Return "No containers" if `Stack` is empty

```
OOLU2468135
CMAU7654321
MSKU1234567
```

[4]

Task 3.3

The file `container_data.csv` store data to test your program. In addition to the `container_number`, `category_of_goods` and `cargo_weight`, the first field states whether a new `ContainerNode` is being added to the `Stack` (INBOUND) or a specific `ContainerNode` is being removed from the `Stack` (OUTBOUND).

Write program code to:

- create two `Stack` instances, one designated as the `mainStack`, and the other as `spareStack`
- process each record in the `container_data.csv` file
- for INBOUND containers, invoke the appropriate methods to insert the new container into the `mainStack` and show all the containers' names in the `mainStack`
- for OUTBOUND containers, first verify whether the topmost container is the correct container to be extracted. If not, extract the topmost container from the `mainStack` and insert it into the `spareStack`. Continue this procedure until the correct container is located at the top of the `mainStack`. Remove the container and return the containers stored in the `spareStack` back into the `mainStack` until no more containers are left in the `spareStack`. Display the containers' names in both `mainStack` and `spareStack` during each operation.

[9]

Save your Jupyter Notebook for Task 3.